

AI-Driven Crime Analytics and Visualization Platform

KSP Datathon 2026 - Challenge 2 - Detailed Project Plan

1. Executive Summary

This document is the complete build plan for Challenge 2 of the KSP Datathon 2026: an AI-Driven Crime Analytics and Visualization Platform for the Karnataka State Police (KSP) and the State Crime Records Bureau (SCRB). The platform unifies fragmented crime records into a live, drillable decision-support system that replaces manual monthly reporting, exposes cross-district offender patterns, forecasts regional risk, and does so with explainability and bias-auditing built in from the start.

The jury for this datathon is composed of serving police and SCRB officers, not machine-learning researchers. That single fact reframes the entire design: usability, trust, auditability, and demonstrable time-savings outweigh model sophistication. Every architectural decision in this plan is made against that scoring function.

The winning thesis in one line

- Challenge 2 is technically low-risk, which makes it high presentation-risk: everyone's dashboard will work, so we do not win on the fact that it runs. We win on cross-district entity resolution as the hook, SHAP-explained risk as the trust story, and automated PDF briefs as the return-on-investment closer.

2. Problem Statement

2.1 Formal statement

Build an AI-powered analytics and visualization platform that unifies fragmented crime records into interactive geospatial dashboards with district-level drilldowns; detects hotspots and anomalies automatically; tracks repeat offenders via entity resolution and network/link analysis; correlates socio-economic indicators with crime patterns; and produces predictive risk scores per region, turning static monthly reports into a live decision-support system.

2.2 Elaborated problem: the real-world context

Karnataka's crime data is generated across 1,100-plus police stations and flows into CCTNS (Crime and Criminal Tracking Network and Systems), a Mission Mode Project under the National e-Governance Plan. CCTNS holds the raw material (FIRs, arrests, chargesheets, dispositions) but the data is siloed by district and by system: FIR data, arrest data, and conviction data commonly live in separate stores that are never joined. The result is that the whole is far less than the sum of its parts.

The Monthly Crime Review that KSP publishes is, today, compiled substantially by hand: officers assemble Excel and PDF summaries manually each month. This is weeks of recurring clerical labour that produces a static, backward-looking snapshot. By the time command staff read it, it describes a world that has already changed.

Four concrete operational failures follow from this, and the platform targets each one directly:

- **Manual reporting burden:** the monthly review is clerical work done by hand; a live dashboard with scheduled auto-generated briefs reclaims that time and makes the picture continuous rather than monthly.
- **Cross-district blindness:** a single burglary gang operating across Bengaluru, Tumakuru, and Mysuru appears in the records as three unrelated local problems. Without deliberate entity resolution and link analysis, nobody connects the dots.
- **Reactive deployment:** patrol allocation follows last month's numbers instead of next week's risk. Hotspot forecasting shifts resource planning from reactive to proactive.
- **No socio-economic context:** crime spikes correlate with festivals, paydays, unemployment, and liquor-outlet density, but police analysts have no tooling to correlate open datasets such as Census indicators against crime data.

One risk is inherent to this problem class and must be addressed openly, not hidden. Predictive policing has a documented failure mode: when a model is trained on recorded crime (arrests, reports) rather than true underlying crime, sending more patrols to a flagged area generates more recorded incidents there, which feeds back into the model and reinforces its focus on that area. This is a runaway feedback loop that entrenches over-policing regardless of the true crime rate. Naming this in front of a police jury, and shipping an explicit bias audit and human-in-the-loop policy, is a maturity signal almost no student team brings.

2.3 Users and what each one needs

Role	Primary need	What the platform gives them
Station officer (SHO)	Local situational awareness	District/station drilldown, offender histories, alert feed for own jurisdiction
District SP	Where to deploy this week	Risk forecast map, trend anomalies, hotspot clusters, repeat-offender networks

Role	Primary need	What the platform gives them
Command staff (IGP/DGP)	Live state-wide picture	State map, cross-district patterns, one-click monthly brief
SCRB analyst	Deep analysis and QA	Entity-resolution review, socio-economic correlations, bias-audit dashboard

3. Solution Architecture: Layer by Layer

The platform is built as five cooperating layers, each with a single responsibility. Data flows upward: raw records are ingested and cleaned at the bottom, transformed into resolved entities and analytical features in the middle, served through a secured API, and consumed by an interactive frontend at the top. A cross-cutting governance layer (auth, audit, bias) touches every level.

3.1 Data layer: ingestion and the synthetic corpus

Real SCRB data will not be available until (and unless) the team is shortlisted, so the data layer is built around a well-documented synthetic generator. This is not a shortcut: a calibrated, documented synthetic dataset is itself a judged deliverable and demonstrates that the team understands the domain's real distributions.

- Synthetic FIR, arrest, and disposition generator calibrated to public statistics. KSP monthly district-wise crime reviews and NCRB Crime in India tables give realistic per-crime-head distributions, seasonality, and district weightings.
- Public open-data loaders: NCRB crime-head tables, Census district indicators, and a festival/payday/rainfall calendar, forming the socio-economic context layer.
- Geo-tagging of incidents to real Karnataka district and synthetic station coordinates so hotspot clustering and maps are meaningful.
- A methodology document explaining exactly how the generator was calibrated. This is what turns fake data into defensible synthetic data for the jury.

3.2 ETL and entity-resolution layer

This is where the platform's signature capability lives. Raw records from multiple silos are cleaned, normalized, and, critically, de-duplicated into resolved persons. The same accused appears across districts under slightly different name spellings, ages, and father's-name variants; probabilistic record linkage collapses these into a single canonical identity.

- A pandas and GeoPandas ETL pipeline: clean, normalize schema, standardize dates and crime heads across sources.
- Entity resolution with Splink (probabilistic record linkage) or the recordlinkage library: fuzzy match on name, age, father's name, and locality, producing match scores rather than brittle exact joins.
- A canonical person store with an audit trail of which raw records were merged, so an analyst can always inspect and override a merge (human-in-the-loop).
- Output loaded into PostgreSQL with PostGIS so downstream geo and analytical queries are fast.

3.3 Analytics engine

Six analytical modules run over the resolved data. Each is deliberately built on classical, explainable methods, with no black-box deep learning, because every output must be defensible to an officer.

Module	Method	Output
Hotspots	DBSCAN and KDE over geo-tagged incidents, per crime type per time window	Density surfaces and cluster labels on the map
Trends and anomalies	STL decomposition and z-score flags per district/crime head	Trend lines with automatic anomaly markers
Repeat offenders	Resolved-entity offense histories and recidivism scoring	Ranked offender lists with cross-district timelines
Network and link analysis	Co-accused and co-location graphs, Louvain community detection	Interactive offender-network graph
Socio-economic correlation	Regression of crime rates against Census and calendar indicators	Correlation cards such as crime up near festivals and paydays
Predictive risk	XGBoost per district by crime-type by week, time-based CV	Per-region risk scores with SHAP explanations

3.4 Backend, API, and governance layer

A FastAPI service exposes analytics endpoints behind JWT authentication and role-based access control (RBAC). Governance is not an add-on: every request is authenticated, authorized by role, row-filtered to the caller's jurisdiction, and written to a tamper-evident audit log.

- FastAPI with JWT auth and RBAC dependency injection, so an SHO sees only their district while SCRB HQ sees all.
- Analytics endpoints (hotspots, trends, network, risk, correlations) plus an alert-feed endpoint and a scheduled-brief generator.
- Hash-chained audit log recording user, role, endpoint, parameters, result hash, and timestamp, each entry chained to the previous so tampering is detectable.
- A bias-audit endpoint that runs disparate-impact checks on the risk model and surfaces them to admins.

3.5 Presentation layer

A React frontend delivers a map-first, drillable experience: state map to district to station, with timelines, network graphs, an automated alert feed, and one-click PDF briefs. The design language is clean, government-appropriate, and legible on a projector during a live demo.

4. Self-Learning and Self-Correcting Backend

You asked specifically for a backend that learns and corrects itself. In a safety-critical policing context this must be done carefully: an uncontrolled self-updating model is exactly what creates the feedback loop described earlier. The design below gives genuine adaptivity while keeping a human in the loop and the whole process auditable.

4.1 Closed-loop retraining with guardrails

- Scheduled retraining: the risk model retrains on a rolling window with strict time-based cross-validation, so it adapts to new patterns without leaking future data into the past.
- Champion and challenger promotion: a newly trained model is a challenger evaluated against the live champion on held-out recent data. It is promoted only if it beats the champion and passes the bias audit. Promotion is logged and reversible.
- Drift detection: monitor input-feature distributions and prediction distributions; when drift crosses a threshold, flag for retraining rather than silently continuing.

4.2 Feedback capture that avoids the runaway loop

The key insight from the predictive-policing literature is that training on discovered crime (where patrols happened to go) corrupts the model. The platform breaks this loop deliberately:

- Officers rate each alert or forecast as useful or not. This human feedback becomes a supervised signal for ranking, separate from raw incident counts.
- The risk model is weighted toward victim-reported and citizen-reported crime rather than patrol-discovered arrests, reducing the self-reinforcing signal.
- A counterfactual check: the bias audit continuously compares the predicted-risk distribution against protected-attribute and jurisdiction distributions and blocks promotion if disparate impact worsens.

Design principle

- Self-learning here means human-supervised, audited, reversible adaptation, never an autonomous model that decides where to send officers on its own. State this explicitly to the jury; it is the difference between naive AI and responsible AI.

5. Interactive Frontend: Design and Screens

The frontend is the demo. Because the jury will judge much of the platform through what they see on screen, the interface is designed to be immediately legible, visually confident, and fast to drill through. The design system uses a clean government-appropriate palette (deep navy, slate, and a single accent), generous whitespace, a large map canvas, and consistent card components.

5.1 Stack

- React with TypeScript and a Vite build.
- Mapping: Leaflet or Kepler.gl for the geospatial canvas, with H3 hex-bin aggregation for smooth zoomed-out density.
- Charts: Recharts or Chart.js for trends and correlation cards.
- Network graph: Cytoscape.js for the offender-network view.
- State and data: React Query against the FastAPI endpoints, with role-aware routing.

5.2 Core screens

Screen	Purpose	Signature element
State Map	Entry point, live state-wide picture	Choropleth plus hotspot layer; click a district to drill
District Drilldown	Local detail for SP and SHO	Station breakdown, trend sparklines, anomaly flags

Screen	Purpose	Signature element
Offender Network	Cross-district link analysis	Interactive graph, communities colour-coded
Risk Forecast	Where to deploy next week	Ranked regions with a SHAP why panel per score
Trends	Time-series exploration	STL trend and anomaly markers, crime-head filters
Alert Feed	Proactive notifications	Ranked alerts, one-click useful/not feedback
Monthly Brief	The ROI closer	One-click auto-generated PDF review
Bias and Audit	Governance and admin	Disparate-impact dashboard and audit-log viewer

6. Wireframes

The two most important screens are described below in layout terms. The State Map is the entry point and first impression; the Risk Forecast screen carries the explainability story that wins trust.

6.1 State Map (landing screen)

Reading top to bottom, the screen has a slim top bar carrying the KSP identity on the left, a global time-window and crime-head filter in the centre, and the role badge with user menu on the right. Below the top bar the screen divides into three columns.

- Left rail (narrow): layer toggles for hotspots, choropleth, and offender density, plus a live top-movers list of districts trending up.
- Centre (dominant): the Karnataka map canvas showing a choropleth with a hotspot heat layer; clicking any district drills into the District screen.
- Right panel (collapsible): state-wide summary cards for total incidents, week-over-week change, and active alerts.

6.2 Risk Forecast (the trust screen)

The screen splits left and right. The left two-thirds is a ranked list of regions for the coming week; each row shows the region name, a risk score, and a small trend arrow. Selecting a row opens the right one-third: a SHAP why-this-score panel.

- Left two-thirds: ranked regions for the week ahead, each with a risk score and trend direction, sortable and filterable by crime head.
- Right one-third: the SHAP panel listing the top contributing factors in plain language (for example festival proximity, recent payday, last-month trend, prior recidivism density) with signed contributions.
- A persistent banner reminds the user that scores are advisory and human-reviewed, reinforcing the human-in-the-loop policy.

7. System Diagrams

7.1 End-to-end data and control flow

The pipeline is a single upward flow from raw data to the officer's screen. Each stage below feeds the next:

- Sources: public data (KSP monthly reviews, NCRB tables, Census indicators, festival/payday calendar) plus the synthetic FIR/arrest/disposition generator.
- ETL pipeline (pandas and GeoPandas): clean, then normalize, then geo-tag.
- Entity resolution (Splink or recordlinkage): fuzzy match on name, age, and father's name into canonical persons, with a merge audit and analyst override.
- Storage: PostgreSQL with PostGIS as the single analytical store.
- Analytics engine: hotspots, trends and anomalies, recidivism scoring, network and link analysis, socio-economic correlation, and predictive risk with SHAP.
- API: FastAPI with JWT and RBAC row filters; every request is hash-chained into the audit log.
- Frontend: the React map-first app, from State Map to District to Station, plus Network, Risk, Trends, Alert Feed, Monthly Brief, and Bias/Audit.

7.2 Self-learning control loop

The adaptivity loop is a closed cycle with a human checkpoint and two safety gates:

- The live champion model produces alerts and risk scores shown to officers.
- Officers rate each output as useful or not; this feedback flows into a feedback store alongside victim-reported weighting.
- A challenger model retrains on a rolling window using time-based cross-validation, informed by the feedback store.
- A drift detector watches feature and prediction distributions and triggers retraining when drift crosses a threshold.
- The challenger is promoted to champion only if it beats the current champion and passes the bias audit; promotion is logged and reversible.

8. Folder and File Architecture

The repository is organized by layer, mirroring the architecture above. Each top-level directory maps to one responsibility, which keeps the codebase navigable for a team working in parallel. The tree below uses indentation for nesting.

crime-analytics-platform/
docker-compose.yml postgres+postgis, backend, frontend, scheduler
README.md setup, demo script, screenshots
etl/
generators/synthetic_fir.py calibrated synthetic FIR/arrest/disposition generator
loaders/ksp_monthly.py KSP monthly-review parser
loaders/ncrb.py NCRB crime-head tables
loaders/census.py Census + festival/payday/rainfall calendar
entity_resolution.py Splink / recordlinkage person dedup
pipeline.py orchestrated ETL into PostgreSQL/PostGIS
analytics/
hotspots.py DBSCAN + KDE surfaces
trends.py STL + anomaly flags
recidivism.py repeat-offender scoring
network.py link analysis, Louvain communities
correlation.py socio-economic regressions
risk_model/train.py XGBoost + time-based CV
risk_model/explain.py SHAP values per prediction
risk_model/promote.py champion/challenger promotion gate
risk_model/drift.py feature/prediction drift detector
risk_model/bias_audit.py disparate-impact checks (signature module)
backend/app/
main.py FastAPI entry
api/ auth, analytics, network, alerts, briefs, audit
core/security.py JWT, RBAC dependencies
core/audit.py hash-chained audit log
services/ brief generator (PDF), alert ranking, feedback store
models/ SQLAlchemy: crimes, persons, cases, users, audit, feedback
frontend/src/
pages/ StateMap, DistrictDrilldown, OffenderNetwork,
RiskForecast, Trends, AlertFeed, MonthlyBrief, BiasAudit
components/ MapView, GraphView, ShapPanel, TrendChart, AlertCard
hooks/ useAuth, useRole, data-fetch hooks
i18n/ EN / KN interface strings
eval/
model_report.py risk-model accuracy + calibration report
bias_report.py disparate-impact report for the slide
docs/
methodology.md synthetic-data calibration, model choices
ethics_and_bias.md bias mitigation, human-in-the-loop policy

9. Expected Output

At the end of the build the team ships a running, demonstrable platform plus a small set of documents that convert engineering into evidence. The concrete deliverables are:

- A live web application (Dockerized) with the eight screens in section 5, seeded with the calibrated synthetic dataset.
- Cross-district offender resolution demonstrated end-to-end: three seemingly separate local cases collapsed into one offender network on screen.
- A predictive risk map for the week ahead, with a SHAP why panel on every score.
- An automated PDF monthly brief generated with one click, replacing the manual review.
- An alert feed with officer feedback capture wired into the self-learning loop.
- A bias-audit dashboard and an ethics_and_bias.md document with an explicit human-in-the-loop policy.
- A model report with an accuracy and calibration number to put on the pitch slide.
- A methodology.md documenting the synthetic-data calibration, a judged deliverable in its own right.

10. Suggested Improvements and Advancements

The following go beyond the baseline brief. Each is chosen for high jury impact relative to build cost; most are additive modules that do not risk the core.

10.1 High-impact, low-cost (build these)

- Kannada interface toggle (i18n): even without full NLP, a Kannada-labelled UI signals adoption-readiness to a Karnataka jury and is cheap to add.
- A saved-clerical-hours counter: quantify and display the estimated hours the automated brief saves per month, a concrete ROI number officers remember.
- Natural-language query lite: a small text-to-filter box (for example, show chain-snatching in Mysuru in the last three months) that maps to dashboard filters, borrowing Challenge 1's wow without its full complexity.
- Confidence bands on every forecast: show uncertainty, not just a point estimate. This reads as scientific honesty to a technical judge.

10.2 Ambitious (if time permits)

- Offline or on-premise deployment mode: police data cannot leave government infrastructure, so demonstrating a fully self-hosted mode is a strong credibility point.
- Patrol-optimization suggestion: given risk scores and available units, suggest a deployment plan, framed strictly as advisory and human-approved.
- A what-if simulator: let command staff simulate the effect of adding patrols to a district on projected risk.
- Explainable anomaly narratives: auto-generate a one-sentence plain-language explanation for each flagged anomaly.

10.3 Governance advancements (your differentiator)

- Continuous disparate-impact monitoring with automatic promotion-blocking, as described in section 4.
- A full audit-log export and verification tool that proves the hash chain is intact.
- A published model card documenting intended use, limitations, and known failure modes.

11. Roadmap (5 Weeks)

Effort is sequenced by demo value, not by architectural order. The goal is to always have a demoable product and to finish the trust-building modules that disproportionately win with this jury.

Week	Milestone	Why it is prioritized here
1	Data, ETL, and entity resolution	Powers the headline cross-district demo; everything sits on this data
2	Hotspots and district drilldown map	The expected core; get it working, do not over-polish
3	Network analysis and recidivism scoring	The cross-district story becomes visual and interactive
4	Risk model, SHAP, and bias audit	The defensibility and trust anchor
5	Dashboard polish, alerts, PDF briefs, docs	The ROI closer, ethics doc, and demo rehearsal

12. Risks and Mitigations

Risk	Severity	Mitigation
------	----------	------------

Risk	Severity	Mitigation
Just-another-dashboard perception	High	Lead with entity resolution, SHAP, and bias, not the map
Entity-resolution false merges	Medium	Probabilistic scores with analyst override, never silent auto-merge
Predictive-policing bias feedback loop	High	Victim-reported weighting, feedback signal, promotion-blocking audit
Synthetic data looks unrealistic	Medium	Calibrate to NCRB and KSP distributions; document in methodology.md
Demo fails live	High	Dockerized, seeded, rehearsed; screenshots as fallback in README

The one thing to rehearse

- Feasibility is very high, so you cannot win on the fact that it works, because everyone's will. Rehearse the narrative as hard as you build the code: entity resolution as the hook, SHAP and bias as the trust story, and the auto-generated brief as the return-on-investment closer.